Projet web statique intégré

Informatique — Licence 1 (L1)

Cours INF-L1-22

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Conduire un projet web complet, du besoin au site en ligne : rédiger un cahier des charges, dessiner les maquettes, intégrer les pages en HTML, CSS et JavaScript, tester, publier et présenter son travail comme un professionnel.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Rédiger le cahier des charges d'un site vitrine
• Leçon 2 — Arborescence et maquettes des pages
• Leçon 3 — Intégration HTML des différentes pages
• Leçon 4 — Mise en forme CSS et adaptation aux mobiles
• Leçon 5 — Touches d'interactivité en JavaScript
• Leçon 6 — Tests, corrections et mise en ligne du site
• Leçon 7 — Présentation et soutenance du projet
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

Jusqu'ici, tu as appris les langages un par un : HTML pour structurer (INF-L1-04), CSS pour mettre en forme (INF-L1-05), JavaScript pour interagir (INF-L1-06). Mais un vrai site ne naît pas en ouvrant directement son éditeur de code : il naît d'un besoin, exprimé par une personne réelle qui attend un résultat utile. Ce cours-projet t'apprend la démarche complète d'un professionnel : comprendre le besoin, concevoir, réaliser, vérifier, livrer — dans cet ordre, et jamais dans l'autre sens.

Tout le cours suit un fil rouge unique : le site vitrine du restaurant fictif « Saveurs du Fleuve », situé à l'ACI 2000, à Bamako. Sa gérante veut présenter ses plats, ses horaires et recevoir des appels de réservation. Chaque leçon fait avancer CE projet, avec des documents réels : le cahier des charges rempli, l'arborescence, les maquettes, le code des pages, la grille de tests et le plan de soutenance. À la fin, tu sauras refaire la même démarche pour n'importe quel commerce de ton quartier.

Ce projet est aussi ta première expérience défendable en entretien : un site en ligne, une adresse à montrer, des choix à expliquer. Travaille donc comme si le client existait vraiment : note tes décisions, respecte les délais que tu te fixes, et garde chaque version. La technique représente la moitié du métier ; l'autre moitié, c'est la méthode — et c'est elle que ce cours t'enseigne.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Recueillir le besoin d'un client et le transformer en cahier des charges clair et complet.
- Dessiner l'arborescence d'un site et les maquettes fil de fer de ses pages.
- Intégrer plusieurs pages HTML cohérentes à partir des maquettes validées.
- Mettre en forme le site en CSS et l'adapter aux écrans de téléphone (responsive).
- Ajouter des interactions JavaScript utiles et robustes (menu mobile, validation de formulaire).
- Tester méthodiquement un site, corriger les défauts et le publier en ligne gratuitement.
- Présenter et défendre son projet à l'oral avec démonstration, bilan et perspectives.

Leçon 1 — Rédiger le cahier des charges d'un site vitrine

Avant la première ligne de code, le développeur professionnel pose des questions. Le cahier des charges est le document qui transforme une demande floue (« je veux un site pour mon restaurant ») en un engagement précis : qui est le client, quel est l'objectif du site, qui va le visiter, quelles pages il contiendra, quels contenus le client fournira, quelles contraintes respecter et à quelle date livrer. Sans ce document, chacun imagine un site différent — et les déceptions arrivent à la livraison, quand il est trop tard pour changer sans tout refaire.

Pour recueillir ces informations, on mène un court entretien avec le client, en préparant ses questions à l'avance : Que doit faire le visiteur sur le site (appeler, réserver, commander, simplement découvrir) ? Quelles informations sont indispensables (adresse, horaires, prix) ? Le client a-t-il déjà un logo, des photos, des textes ? Un site vitrine ne vend pas en ligne : il présente et oriente vers un contact. Pour notre fil rouge, Mme Fatoumata Diallo, gérante de « Saveurs du Fleuve », veut que le visiteur trouve la carte et le numéro de téléphone en moins de trois clics, depuis un téléphone.

Un bon cahier des charges se termine par des critères de réussite mesurables : pas « un beau site », mais « le site s'affiche correctement sur un téléphone Android d'entrée de gamme », « la page Contact contient le numéro, l'adresse et les horaires », « le site est en ligne à telle date ». Ces critères serviront de liste de contrôle à la livraison et de protection pour les deux parties : le client sait ce qu'il reçoit, le développeur sait ce qu'il doit produire — ni plus, ni moins.

L'essentiel
Le cahier des charges écrit, avant tout code, l'objectif du site, son public, ses pages, les contenus à fournir, les contraintes, le délai et les critères de réussite mesurables. Il protège le client comme le développeur.

Exemple — Extrait du cahier des charges de « Saveurs du Fleuve »

```
CAHIER DES CHARGES — SITE VITRINE « SAVEURS DU FLEUVE »
Client : Mme Fatoumata Diallo, restaurant, ACI 2000, Bamako
Objectif : présenter le restaurant et recevoir des
réservations par téléphone et WhatsApp.
Public visé : habitants de Bamako, familles, employés
des bureaux du quartier, souvent sur téléphone.
Pages : Accueil, Nos plats, À propos, Contact.
Contenus fournis par la cliente : logo, 12 photos de
plats, carte du menu avec prix en FCFA, horaires.
Charte : rouge brique #A63D2A, jaune safran #E9B44C,
fond crème #FFF9F2, titres en Georgia.
Contraintes : français uniquement, lisible sur mobile,
aucun paiement en ligne (site vitrine uniquement).
Délai : 3 semaines. Outils gratuits uniquement.
Critère de réussite : numéro et carte accessibles en
moins de 3 clics depuis la page d'accueil.
```

À retenir

- On code après avoir compris le besoin, jamais avant : l'entretien client précède toujours l'éditeur de texte.
- Un objectif flou (« un beau site ») ne se teste pas ; un critère mesurable (« numéro visible en 3 clics ») si.
- Le cahier des charges liste aussi ce que le CLIENT doit fournir (photos, textes, prix) et pour quand.

Exercice

Un ami commerçant te dit : « Fais-moi un site pour ma boutique, quelque chose de moderne, tu vois ce que je veux dire. » Rédige les huit questions précises que tu lui poseras avant d'accepter le travail, et explique pour chacune ce qu'elle permettra de décider.

✔ Corrigé

Exemples de questions attendues : 1) Que vends-tu exactement et à qui ? (définit le public et les pages). 2) Que doit pouvoir faire le visiteur : appeler, écrire sur WhatsApp, commander ? (définit les actions et les boutons). 3) As-tu un logo, des photos de tes produits et tes prix à jour ? (définit les contenus à collecter). 4) Quelles couleurs ou quel style te plaisent, avec un exemple de site que tu aimes ? (définit la charte). 5) Le site doit-il être en français seulement ? (définit les langues). 6) Pour quand le veux-tu ? (définit le délai et le planning). 7) Qui validera les textes et les photos avant publication ? (évite les allers-retours infinis). 8) Après la livraison, qui mettra les prix à jour et comment ? (prévoit la maintenance). Sans ces réponses, « moderne » peut signifier dix sites différents : le cahier des charges sert justement à n'en construire qu'un — le bon.

Leçon 2 — Arborescence et maquettes des pages

Une fois le besoin écrit, on dessine le plan du site avant de le construire, comme l'architecte dessine la maison avant de couler le béton. L'arborescence est la carte des pages et de leurs liens : elle montre la page d'accueil, les pages principales et, si

besoin, les sous-pages. On la dessine en texte ou avec des boîtes reliées par des flèches. Une règle d'or : depuis l'accueil, toute information importante doit être atteignable en trois clics au maximum. Si la carte du restaurant se cache au quatrième niveau, les visiteurs ne la trouveront jamais.

Ensuite, pour chaque page, on réalise une maquette « fil de fer » (wireframe) : un schéma en noir et blanc, sans couleurs ni belles images, qui place les blocs — en-tête, menu, image principale, texte, cartes, pied de page — et indique leur contenu réel sous forme de titres et de courtes notes. La maquette se fait au crayon sur papier ou dans un outil comme Figma. Son intérêt : elle se modifie en deux minutes, là où déplacer un bloc dans du code déjà écrit prend une heure. On la fait valider par le client avant de coder.

La maquette doit être pensée d'abord pour le téléphone : c'est l'écran de la majorité des visiteurs à Bamako. On dessine donc la version mobile (une seule colonne, menu compact dit « burger ») puis la version ordinateur (colonnes côte à côte). Ce réflexe « mobile d'abord » évite le piège classique : un site magnifique sur grand écran, illisible sur le téléphone des clients. Pour « Saveurs du Fleuve », la maquette de l'accueil place la photo, le slogan et le bouton « Voir nos plats » dès le premier écran, puis les plats signatures et les informations pratiques.

L'essentiel
L'arborescence fixe la liste des pages et leurs liens (règle des 3 clics) ; la maquette fil de fer place les blocs de chaque page, en noir et blanc, mobile d'abord. On valide les maquettes AVANT de coder : modifier un dessin coûte des minutes, modifier du code coûte des heures.

Exemple — Arborescence et maquette mobile de l'accueil

```
ARBORESCENCE DU SITE « SAVEURS DU FLEUVE »
Accueil (index.html)
|-- Nos plats (plats.html)
| -> photo, description et prix en FCFA de chaque plat
|-- À propos (a-propos.html)
| -> histoire du restaurant, équipe, photos des lieux
|-- Contact (contact.html)
-> adresse, horaires, téléphone, formulaire

MAQUETTE FIL DE FER — ACCUEIL (version mobile)
+----------------------------------+
| LOGO [ = Menu ] |
+----------------------------------+
| PHOTO DU RESTAURANT |
| « La cuisine malienne comme |
| à la maison » |
| [ Voir nos plats ] |
+----------------------------------+
| PLATS SIGNATURES |
| [photo plat 1] nom — prix |
| [photo plat 2] nom — prix |
| [photo plat 3] nom — prix |
+----------------------------------+
| TÉMOIGNAGE D'UNE CLIENTE |
+----------------------------------+
| Horaires • Adresse • Téléphone |
| © Saveurs du Fleuve |
+----------------------------------+
```

À retenir

• Arborescence = quelles pages, quels liens ; maquette = quels blocs, à quelle place, dans chaque page.

• Une maquette fil de fer ne contient ni couleurs ni vraies photos : on discute la structure, pas les goûts.

• Dessiner la version mobile en premier : c'est l'écran réel de la plupart des visiteurs.

Exercice

Pour la page « Nos plats » de « Saveurs du Fleuve », dessine la maquette fil de fer en version mobile (format texte comme dans l'exemple) : elle doit permettre de voir, pour au moins six plats, la photo, le nom, une courte description et le prix, plus un bouton pour appeler le restaurant. Explique tes choix d'ordre d'affichage.

✔ Corrigé
Maquette attendue : en-tête avec logo et menu (identique à l'accueil, pour ne pas perdre le visiteur) ; titre de page « Nos plats » ; éventuellement des filtres simples par catégorie (Entrées, Plats, Grillades, Boissons) ; puis la liste des plats en une colonne : pour chaque plat, un bloc [photo] / nom en gros / description en une ligne / prix en FCFA bien visible. Les plats les plus demandés (poulet yassa, tieboudienne) arrivent en premier : on place en haut ce qui fait vendre. En bas de page, un bloc fixe d'appel à l'action : « Commander ou réserver : [Appeler] [WhatsApp] », car après avoir regardé les plats, le visiteur doit pouvoir agir sans remonter toute la page. Le pied de page reste identique aux autres pages : horaires, adresse, téléphone. Choix justifiés : une colonne (lecture mobile), prix toujours visibles (question n° 1 des clients), bouton d'appel en fin de parcours (conversion).

Leçon 3 — Intégration HTML des différentes pages

L'intégration est le moment où les maquettes deviennent de vraies pages. On commence par organiser le dossier du projet de façon propre : un fichier HTML par page à la racine, un dossier css pour la feuille de style unique, un dossier js pour le script, un dossier images pour les photos. Cette organisation n'est pas un caprice : elle permettra la mise en ligne en un seul envoi, et n'importe quel autre développeur comprendra ton projet en dix secondes. La page d'accueil se nomme toujours index.html : c'est celle que les hébergeurs servent par défaut.

Chaque page reprend la structure sémantique vue en INF-L1-04 : un header avec le logo et la navigation, un main avec le contenu propre à la page, un footer commun. L'en-tête et le pied de page sont identiques sur toutes les pages : on les écrit une fois, puis on les copie en ne changeant que le titre de l'onglet et le contenu central. Cette répétition assumée garantit au visiteur une navigation stable — le menu ne doit jamais « bouger » d'une page à l'autre. Pense aussi aux attributs alt des images : ils servent les malvoyants, le référencement, et affichent un texte si une photo ne charge pas sur une connexion lente.

On intègre page par page, dans l'ordre de l'arborescence, en vérifiant chaque page dans le navigateur dès qu'elle est terminée : liens cliqués un par un, images affichées, titres hiérarchisés (un seul h1 par page). Le contenu réel — textes de la cliente, prix définitifs — remplace les textes d'attente dès cette étape ; laisser du « lorem ipsum » à la livraison est une faute professionnelle. Pour « Saveurs du Fleuve », les quatre pages partagent la même navigation et ne diffèrent que par leur main : accueil, liste des plats, histoire du restaurant, coordonnées et formulaire.

L'essentiel
Dossier propre (pages à la racine, dossiers css, js, images), une feuille de style commune, header et footer identiques partout, contenu réel dès l'intégration et chaque page testée dans le navigateur au fur et à mesure.

Exemple — Squelette de la page d'accueil (index.html)

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<title>Saveurs du Fleuve — Restaurant malien à Bamako</title>
<meta name="description" content="Restaurant malien à
l'ACI 2000, Bamako : plats traditionnels,
grillades et jus frais.">
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<header>
<img src="images/logo.png" alt="Logo Saveurs du Fleuve">
<button class="bouton-menu" aria-label="Ouvrir le menu">□</button>
<nav>
<ul>
<li><a href="index.html">Accueil</a></li>
<li><a href="plats.html">Nos plats</a></li>
<li><a href="a-propos.html">À propos</a></li>
<li><a href="contact.html">Contact</a></li>
</ul>
</nav>
</header>
<main>
<section class="hero">
<h1>La cuisine malienne comme à la maison</h1>
<p>Tieboudienne, yassa, mafé… préparés chaque jour.</p>
<a class="bouton" href="plats.html">Voir nos plats</a>
</section>
<section class="plats-signatures">
<h2>Nos plats signatures</h2>
<!-- 3 cartes de plats insérées ici -->
</section>
</main>
<footer>
<p>ACI 2000, Bamako — Ouvert tous les jours 11h à 23h</p>
<p>Tél. : +223 70 12 34 56 — © <span id="annee">2026</span>
Saveurs du Fleuve</p>
</footer>
<script src="js/script.js"></script>
</body>
</html>
```

À retenir

• index.html est la porte d'entrée du site ; les autres pages portent des noms courts, sans espaces ni accents.

• Header, navigation et footer identiques sur toutes les pages : la stabilité rassure le visiteur.

• Contenu réel tout de suite (vrais textes, vrais prix) : jamais de lorem ipsum dans une livraison.

Exercice

La page plats.html de « Saveurs du Fleuve » affiche six plats. Décris la structure HTML d'UNE carte de plat (photo, nom, description, prix) en utilisant des balises sémantiques et des classes réutilisables, puis explique comment la même structure sert pour les six plats et ce qu'il faudra changer dans le CSS si la cliente ajoute un septième plat.

✔ Corrigé
Chaque plat tient dans un <article class="carte-plat"> contenant : <img src="images/yassa.jpg" alt="Poulet yassa servi avec du riz">, <h3>Poulet yassa</h3>, Poulet mariné au citron et aux oignons… et 2 500 FCFA . Les six plats réutilisent exactement la même structure et les mêmes classes : seul le contenu (photo, nom, texte, prix) change. C'est le principe d'un composant : une seule définition de style dans le CSS (.carte-plat, .carte-plat .prix…) s'applique à tous. Si la cliente ajoute un septième plat, on copie-colle un bloc <article>, on change son contenu, et il n'y a RIEN à modifier dans le CSS : la grille ou la liste l'accueillera automatiquement. C'est pour cela qu'on ne crée jamais de classes du type .plat1, .plat2 : elles obligeraient à écrire du nouveau CSS à chaque ajout.

Leçon 4 — Mise en forme CSS et adaptation aux mobiles

Le CSS du projet se construit en trois couches, dans cet ordre : d'abord les fondations (variables de couleurs issues de la charte, polices, marges remises à zéro, styles généraux du corps de page), ensuite les composants réutilisables (en-tête, menu, boutons, cartes de plats, pied de page), enfin les adaptations par taille d'écran. Travailler dans cet ordre évite de se battre avec des styles contradictoires. Les variables CSS, déclarées dans :root, sont la clé de la cohérence : si la cliente décide que le rouge brique devient bordeaux, on change une seule ligne et tout le site suit.

L'adaptation aux mobiles — le responsive design — repose principalement sur les requêtes média (@media) et sur des mises en page flexibles (Flexbox et Grid, vus en INF-L1-05). La démarche « mobile d'abord » consiste à écrire d'abord les styles pour petit écran (une colonne, textes lisibles sans zoomer, boutons assez grands pour le doigt), puis à enrichir pour les écrans larges avec des requêtes en min-width. Pour « Saveurs du Fleuve », les cartes de plats passent de trois colonnes sur ordinateur à une seule colonne sur téléphone, et le menu horizontal devient un menu burger.

Trois détails séparent un site « qui marche » d'un site professionnel : les images ne dépassent jamais de l'écran (max-width: 100%), aucun défilement horizontal n'apparaît sur téléphone, et les zones cliquables font au moins la taille d'un doigt (environ 44 pixels). On vérifie ces points en réduisant progressivement la fenêtre du navigateur ou avec le mode appareil mobile des outils de développement. Enfin, garde une feuille de style lisible : commentaires par section, noms de classes en français clair (.carte-plat, .bouton-menu) — ton futur toi, dans trois mois, te dira merci.

L'essentiel
Construire le CSS en couches (fondations avec variables, composants, adaptations), penser mobile d'abord, et utiliser les requêtes média pour passer les cartes et le menu d'une disposition large à une disposition en colonne sur téléphone.

Exemple — Fondations et bascule responsive des cartes de plats

```
/* ===== Fondations : charte de Saveurs du Fleuve ===== */
:root {
--brique: #A63D2A;
--safran: #E9B44C;
--fond: #FFF9F2;
--texte: #2B2118;
}
* { box-sizing: border-box; }
body {
margin: 0;
font-family: Arial, Helvetica, sans-serif;
background: var(--fond);
color: var(--texte);
}
img { max-width: 100%; height: auto; }

/* ===== Composant : cartes de plats (mobile d'abord) ===== */
.cartes-plats {
display: flex;
flex-direction: column;
gap: 1rem;
padding: 1rem;
}
.carte-plat {
background: #FFFFFF;
border-top: 4px solid var(--safran);
padding: 1rem;
}
.carte-plat .prix {
color: var(--brique);
font-weight: bold;
font-size: 1.2rem;
}

/* ===== Écrans larges : 3 cartes côte à côte ===== */
@media (min-width: 768px) {
.cartes-plats { flex-direction: row; }
.carte-plat { flex: 1; }
}
```

À retenir

- Mobile d'abord : styles de base pour petit écran, enrichissements en min-width pour les grands.
- Les variables CSS dans :root rendent la charte modifiable en un seul endroit.

• Les trois contrôles express : pas de dépassement d'image, pas de défilement horizontal, boutons assez grands.

Exercice

Sur téléphone, le menu de « Saveurs du Fleuve » doit être masqué par défaut et ne s'ouvrir que via le bouton burger ; sur ordinateur, il doit toujours être visible en ligne. Écris les règles CSS correspondantes (le bouton burger ne s'affiche que sur mobile) et explique le rôle du point de rupture choisi.

✔ Corrigé
Mobile d'abord : .bouton-menu { display: block; } (visible sur téléphone) et nav ul { display: none; flex-direction: column; } avec nav ul.ouvert { display: flex; } — la classe .ouvert sera ajoutée/retirée par JavaScript (leçon 5). Pour les écrans d'au moins 768 px : @media (min-width: 768px) { .bouton-menu { display: none; } nav ul { display: flex; flex-direction: row; } }. Le point de rupture de 768 px correspond à la largeur typique d'une tablette en portrait : en dessous, la place manque pour un menu horizontal lisible ; au-dessus, il tient confortablement. Le choix d'un point de rupture se justifie toujours par le CONTENU (à partir de quelle largeur le menu et les cartes respirent-ils ?), jamais par un modèle de téléphone précis : les appareils changent, le contenu reste.

Leçon 5 — Touches d'interactivité en JavaScript

Un site vitrine n'a pas besoin d'effets partout : il a besoin de quelques interactions qui servent le visiteur. Pour « Saveurs du Fleuve », trois fonctions JavaScript suffisent, toutes vues en INF-L1-06 : ouvrir et fermer le menu mobile, afficher automatiquement l'année courante dans le pied de page, et contrôler le formulaire de contact avant l'envoi (champs obligatoires, numéro de téléphone plausible). Ces trois scripts vivent dans un seul fichier js/script.js, chargé en fin de page par toutes les pages du site — là encore, on écrit une fois, on réutilise partout.

La règle d'or de l'interactivité : le site doit rester utilisable même si JavaScript échoue ou est désactivé. Le menu, par exemple, ne doit pas emprisonner la navigation : c'est pourquoi on masque la liste par CSS uniquement quand on sait que le bouton burger peut l'ouvrir, et on prévoit des liens accessibles. De même, le contrôle du formulaire côté navigateur est un confort (réponse immédiate, message clair en français), pas une sécurité : il guide l'utilisateur, il ne remplace pas la politesse d'un message d'erreur compréhensible.

Chaque interaction se code et se teste isolément : on écrit le menu, on le teste sur téléphone et ordinateur ; puis l'année ; puis le formulaire, en essayant volontairement les cas limites — champ vide, numéro trop court, espaces dans le numéro. Un script qui plante bloque tous les suivants dans le fichier : on garde donc un code simple, commenté, et on surveille la console du navigateur, qui signale les erreurs en rouge. Mieux vaut trois interactions impeccables que dix gadgets bancals : la cliente paie pour un site fiable, pas pour un feu d'artifice.

L'essentiel
Peu d'interactions, mais utiles et testées : menu mobile, année automatique, validation du formulaire. Le site doit rester navigable sans JavaScript, et chaque script se vérifie seul, cas limites compris, console ouverte.

Exemple — script.js : menu mobile, année et contrôle du formulaire

```
// ===== Menu mobile : ouvrir / fermer au clic =====
const boutonMenu = document.querySelector('.bouton-menu');
const menu = document.querySelector('nav ul');

boutonMenu.addEventListener('click', function () {
menu.classList.toggle('ouvert');
});

// ===== Année automatique dans le pied de page =====
const zoneAnnee = document.getElementById('annee');
if (zoneAnnee) {
zoneAnnee.textContent = new Date().getFullYear();
}

// ===== Contrôle du formulaire de contact =====
const formulaire = document.getElementById('form-contact');
if (formulaire) {
formulaire.addEventListener('submit', function (event) {
const tel = document.getElementById('telephone');
const numero = tel.value.replace(/\s/g, '');
if (numero.length < 8) {
event.preventDefault(); // on bloque l'envoi
alert('Merci de saisir un numéro de téléphone valide '
+ '(8 chiffres au minimum).');
tel.focus();
}
});
}
```

À retenir

- Un seul fichier script.js partagé par toutes les pages : corriger une fois, corriger partout.
- Tester les cas limites du formulaire (vide, trop court, espaces) : c'est là que vivent les erreurs.
- Si JavaScript casse, le site doit rester utilisable : l'interactivité est un bonus, pas une condition.

Exercice

La cliente veut un bouton « Appeler » qui n'apparaisse que sur téléphone (sur ordinateur, le numéro s'affiche en texte simple). Sans écrire le code complet, explique la démarche : quel élément HTML créer, comment le masquer/afficher selon l'écran, quel schéma d'URL permet de déclencher l'appel, et comment tu vérifieras que tout

fonctionne.

✔ Corrigé
Démarche : 1) HTML — ajouter dans l'en-tête ou la page Contact un lien Appeler le restaurant ; le schéma tel: transforme le lien en déclencheur d'appel sur téléphone. 2) CSS mobile d'abord — .bouton-appel visible par défaut (petits écrans), puis @media (min-width: 768px) { .bouton-appel { display: none; } } pendant que le numéro en texte, lui, peut être masqué sur mobile et visible sur ordinateur, ou simplement toujours visible dans la page Contact. 3) Aucun JavaScript n'est nécessaire : CSS suffit, c'est la solution la plus robuste. 4) Vérification : réduire la fenêtre du navigateur pour voir le bouton apparaître/disparaître au point de rupture, tester sur un vrai téléphone (le clic doit ouvrir le composeur avec le numéro pré-rempli), et vérifier sur ordinateur que le bouton est absent et le numéro lisible. Moralité : avant d'écrire du JavaScript, toujours se demander si HTML ou CSS ne résout pas le problème plus simplement.

Leçon 6 — Tests, corrections et mise en ligne du site

Un site n'est pas terminé quand il « marche chez moi » : il est terminé quand il a passé une batterie de tests écrits à l'avance. On prépare une grille de tests qui couvre cinq familles : les liens (chaque lien mène à la bonne page, aucun lien mort), le contenu (orthographe, prix à jour, horaires exacts), l'affichage (téléphone, tablette, ordinateur, fenêtre réduite), les interactions (menu, formulaire avec des données correctes ET incorrectes) et les bases techniques (pages validées par le validateur du W3C, images qui s'affichent, poids des photos raisonnable pour les connexions lentes). Chaque test reçoit un verdict : OK ou à corriger.

Les défauts trouvés se corrigent par ordre de gravité, pas par ordre de découverte : d'abord ce qui bloque le visiteur (lien mort, formulaire qui ne part pas), ensuite ce qui trompe (prix faux, mauvaise adresse), enfin ce qui déçoit (alignement, espace de trop). Après chaque série de corrections, on reteste les points corrigés ET leurs voisins — une correction peut casser autre chose. Ce cycle tester-corriger-retester, fait calmement avant la livraison, évite le pire scénario : découvrir les erreurs en même temps que la cliente, pendant la démonstration.

La mise en ligne d'un site statique peut être entièrement gratuite avec GitHub Pages : on crée un compte GitHub, un dépôt public portant le nom du projet, on y envoie tous les fichiers du site en gardant l'arborescence (index.html à la racine), puis on active Pages dans les réglages du dépôt (Settings, Pages, branche main). Quelques minutes plus tard, le site répond à une adresse du type pseudo.github.io/saveurs-du-fleuve/. On reteste alors le site EN LIGNE : c'est là qu'apparaissent les oublis classiques — une image restée sur ton ordinateur, un lien en chemin absolu local, une casse de nom de fichier différente (Plat.JPG contre plat.jpg).

L'essentiel
Grille de tests écrite à l'avance (liens, contenu, affichage, interactions, technique), corrections triées par gravité, retest après correction. Puis mise en ligne gratuite sur GitHub Pages et nouveau test complet du site en ligne, là où surgissent les oublis de fichiers.

Exemple — Grille de tests (extrait) et recette de mise en ligne

```
GRILLE DE TESTS — SAVEURS DU FLEUVE (extrait)
N° | Test | Attendu | OK
1 | Cliquer chaque lien du menu | Bonne page, 0 erreur | [x]
2 | plats.html sur téléphone (360 px) | Cartes en 1 colonne | [x]
3 | Envoyer le formulaire vide | Message d'erreur | [x]
4 | Numéro à 5 chiffres dans le champ | Envoi bloqué | [x]
5 | Valider index.html sur le W3C | 0 erreur | [x]
6 | Faire trouver l'adresse à 3 proches | < 30 secondes chacun | [ ]
7 | Ouvrir le site avec connexion lente | Page lisible < 5 s | [ ]

MISE EN LIGNE — GITHUB PAGES
1. Créer un dépôt public « saveurs-du-fleuve ».
2. Envoyer TOUS les fichiers (index.html à la racine,
dossiers css, js, images conservés).
3. Settings > Pages > branche « main » > Save.
4. Attendre quelques minutes, ouvrir l'adresse :
https://pseudo.github.io/saveurs-du-fleuve/
5. Refaire la grille de tests complète EN LIGNE.
```

À retenir

- « Ça marche chez moi » n'est pas un test : la grille écrite, exécutée case par case, l'est.
- Corriger par gravité décroissante : bloquant d'abord, trompeur ensuite, esthétique en dernier.
- Le site en ligne se reteste entièrement : fichiers oubliés et casses de noms n'apparaissent que là.

Exercice

Après la mise en ligne, la page d'accueil s'affiche sans aucun style et la photo du restaurant manque, alors que tout était parfait en local. Donne les trois causes les plus probables, dans l'ordre où tu les vérifierais, et la manière de confirmer chacune.

✔ Corrigé
Cause 1 — la feuille de style n'a pas été envoyée ou pas au bon endroit : vérifier dans le dépôt que le dossier css contient style.css et que le lien HTML pointe bien sur css/style.css (chemin relatif). Cause 2 — différence de casse dans les noms : l'ordinateur local ignore souvent la casse, le serveur non ; si le fichier s'appelle Style.css et le lien style.css, le serveur répond « introuvable ». On confirme en ouvrant directement l'adresse du fichier CSS dans le navigateur : 404 = fichier absent ou mal nommé. Cause 3 — l'image est restée en local ou son nom contient un espace ou un accent (photo resto.jpg) que l'URL gère mal : renommer en photo-resto.jpg, renvoyer le fichier dans le dossier images, et vérifier l'attribut src. Méthode générale : ouvrir les outils de développement du navigateur, onglet Réseau, recharger : chaque ressource en erreur 404 désigne le coupable — on ne devine pas, on constate.

Leçon 7 — Présentation et soutenance du projet

Un projet non présenté est un projet à moitié livré : la soutenance est le moment où tu transformes des fichiers en preuve de compétence. Devant le jury — ou la cliente — tu racontes une histoire en quatre temps : le problème de départ (une restauratrice invisible sur internet), la démarche (cahier des charges, maquettes, intégration, tests), la démonstration du site en ligne, et le bilan honnête (ce qui fonctionne, les limites, ce que tu ferais ensuite). Dix minutes bien préparées valent mieux qu'une heure d'improvisation : on écrit son plan, on chronomètre, on répète à voix haute au moins deux fois.

La démonstration est le cœur de la soutenance : prépare un parcours de visiteur précis (arriver sur l'accueil, trouver un plat et son prix, appeler depuis la page Contact) et effectue-le sur le site EN LIGNE, pas sur des captures d'écran — tout en ayant les captures prêtes en secours si la connexion fait défaut. Montre aussi le site sur un téléphone ou en fenêtre réduite : prouver le responsive en direct vaut tous les discours. Garde le code pour les questions : le jury demandera peut-être comment tu as fait le menu mobile ou la validation du formulaire ; savoir ouvrir le bon fichier et la bonne ligne montre que le code est bien le tien.

Le bilan doit être honnête et tourné vers l'avenir : un site vitrine statique ne permet ni réservation en ligne ni modification des prix par la cliente sans toucher au code — ce sont ses limites réelles, à annoncer toi-même avant qu'on te les oppose. Propose ensuite les évolutions possibles : formulaire relié à un service d'envoi, page multilingue français-bambara, commande via WhatsApp approfondie, passage à un outil de gestion de contenu. Un étudiant qui connaît les limites de son travail et sait le prolonger inspire confiance : c'est exactement l'impression à laisser en sortant de la salle.

L'essentiel
Soutenance = une histoire en quatre temps (problème, démarche, démonstration en ligne, bilan honnête), préparée, chronométrée et répétée. Prévoir un plan B sans connexion, et annoncer soi-même les limites du site avant les questions.

Exemple — Plan de soutenance chronométré (10 minutes)

```
PLAN DE SOUTENANCE — PROJET « SAVEURS DU FLEUVE »
0:00–1:00 Accroche : le problème de la cliente
(un restaurant introuvable sur internet)
1:00–2:30 Démarche : cahier des charges, arborescence,
maquettes validées avant de coder
2:30–6:30 DÉMONSTRATION du site en ligne :
accueil -> un plat et son prix -> page Contact
-> appel ; puis affichage sur écran de téléphone
6:30–8:00 Choix techniques : HTML sémantique, Flexbox
et mobile d'abord, JavaScript limité à 3 fonctions
8:00–9:00 Difficultés : menu mobile, casse des fichiers
à la mise en ligne ; solutions trouvées
9:00–10:00 Bilan et limites (pas de réservation en ligne,
prix modifiables seulement dans le code),
évolutions proposées. Questions du jury.

SECOURS : captures d'écran des 4 pages + grille de tests
remplie, sur clé USB et sur le téléphone.
```

À retenir

- Préparer, chronométrer, répéter à voix haute : une soutenance se répète comme un examen, pas comme une conversation.
- Démonstration sur le site en ligne, parcours de visiteur préparé, captures de secours prêtes.
- Annoncer soi-même les limites du projet et ses évolutions possibles : c'est une preuve de maturité, pas un aveu de faiblesse.

Exercice

Prépare les réponses aux trois questions de jury suivantes, en deux ou trois phrases chacune, en t'appuyant sur le projet « Saveurs du Fleuve » : 1) « Pourquoi un site statique et pas un site avec base de données ? » 2) « Comment la cliente modifiera-t-elle ses prix ? » 3) « Qu'est-ce qui vous a pris le plus de temps, et pourquoi ? »

✔ Corrigé
1) « Le besoin de la cliente est de présenter ses plats et d'être appelée : aucune donnée n'a besoin d'être enregistrée ni modifiée souvent. Un site statique est donc plus simple, plus rapide, gratuit à héberger et sans maintenance de sécurité ; une base de données serait un coût sans bénéfice pour ce cahier des charges. » 2) « Dans cette première version, les prix sont dans le code HTML des cartes de plats : la cliente me transmet ses changements et je les applique en quelques minutes, ou je forme la cliente à modifier le fichier. Si les changements devenaient hebdomadaires, je proposerais une version 2 avec un outil de gestion de contenu — c'est une des évolutions de mon bilan. » 3) Réponse personnelle mais structurée : nommer UNE vraie difficulté (par exemple le menu mobile qui restait ouvert sur ordinateur), expliquer la cause trouvée (la requête média masquait la liste mais la classe .ouvert gardait la priorité) et la solution (règle spécifique dans la requête grand écran). Le jury ne cherche pas une réponse parfaite : il vérifie que tu comprends ton propre code et que tu sais raconter un problème, une cause, une solution.

Travaux dirigés (TD)

TD 1 — Analyser et compléter un cahier des charges incomplet

Voici la totalité de la demande reçue pour un nouveau projet : « Bonjour, je suis Aminata, j'ai un salon de coiffure à l'Hippodrome. Je veux un site internet joli avec mes photos, pour avoir plus de clientes. C'est urgent. Merci. » 1) Liste tout ce que ce message permet déjà de savoir. 2) Liste au moins dix informations manquantes indispensables avant de coder, regroupées en quatre familles : le commerce, le public et les usages, les contenus, le cadre du projet. 3) Pour trois de ces informations, explique le risque concret si l'on code sans les connaître.

✔ Corrigé du TD 1
1) Ce que l'on sait : le type de commerce (salon de coiffure), le quartier (Hippodrome, Bamako), le prénom de la cliente (Aminata), un objectif général (attirer plus de clientes), la présence de photos, une urgence exprimée. C'est peu : six éléments, dont aucun n'est directement actionnable dans du code. 2) Informations manquantes — Le commerce : nom exact du salon, adresse complète, horaires, prestations et tarifs (tresses, défrisage, soins…), moyens de contact (téléphone, WhatsApp). Le public et les usages : clientes visées (étudiantes, femmes actives ?), action attendue (appeler pour rendez-vous, voir les modèles, trouver le salon), langue(s) du site. Les contenus : logo existant ou à créer, nombre et qualité des photos, qui rédige les textes, photos de clientes (avec leur accord !). Le cadre : délai réel derrière « urgent », budget éventuel, qui valide, qui mettra à jour les tarifs ensuite, exemples de sites aimés ou détestés. 3) Risques si l'on code à l'aveugle : sans les tarifs, on laisse des espaces vides ou on invente des prix — faute professionnelle grave ; sans savoir que la prise de rendez-vous se fait par WhatsApp, on construit un formulaire que personne n'utilisera ; sans la liste des prestations, la page principale peut présenter en vedette un service que le salon ne propose plus. Conclusion : un entretien de trente minutes et un cahier des charges d'une page évitent des semaines de corrections.

TD 2 — Diagnostiquer une intégration défectueuse

La première version du site « Saveurs du Fleuve » présente les défauts suivants, constatés lors des tests : a) le lien « Nos plats » du menu de la page À propos renvoie une erreur 404 ; b) sur téléphone, il faut défiler horizontalement pour lire la page des plats ; c) deux photos de plats ne s'affichent pas, seule une croix apparaît ; d) le formulaire part même quand le champ téléphone est vide ; e) le prix du tieboudienne affiche 3 500 FCFA alors que la cliente avait validé 3 000 FCFA ; f) le menu mobile s'ouvre mais ne se referme plus. Pour chaque défaut : donne la cause probable, la correction, et classe les six corrections par ordre de priorité de traitement en justifiant ton classement.

✔ Corrigé du TD 2

a) Cause probable : faute dans le href de cette page (par exemple plat.html au lieu de plats.html) ou fichier renommé après coup. Correction : corriger le lien, puis retester les menus des QUATRE pages, car l'erreur vient souvent d'une copie du header. b) Cause probable : un élément trop large (image sans max-width: 100%, ou cartes en ligne sans passage en colonne). Correction : appliquer img { max-width: 100%; } et vérifier la requête média des cartes. c) Cause probable : chemin d'image faux ou casse différente (Yassa.jpg contre yassa.jpg) — le serveur distingue les majuscules. Correction : vérifier les fichiers réellement présents dans le dossier images et aligner noms et src au caractère près. d) Cause : le script de contrôle n'est pas chargé sur cette page, ou l'écouteur d'événement contient une erreur (visible dans la console). Correction : vérifier la balise script en fin de page et la console, retester avec champ vide, numéro court et numéro correct. e) Cause : erreur de saisie du contenu — la plus grave tromperie possible pour un visiteur. Correction immédiate : 3 000 FCFA, et relecture complète des prix contre la carte validée par la cliente. f) Cause probable : le gestionnaire de clic ajoute la classe mais le toggle est mal orthographié, ou un second script écrase l'événement. Correction : vérifier classList.toggle('ouvert') et tester ouverture/fermeture cinq fois de suite. Classement par priorité : 1) e (contenu faux = confiance détruite), 2) a (lien principal mort = visite bloquée), 3) d (formulaire inutilisable = réservations perdues), 4) f (navigation mobile dégradée), 5) b (lecture pénible sur mobile), 6) c (esthétique, mais les plats restent décrits). Justification : on traite d'abord ce qui trompe le visiteur, puis ce qui le bloque, puis ce qui le gêne — jamais l'ordre de découverte.

Travaux pratiques (TP)

TP 1 — Réaliser le site vitrine complet d'un commerce de ton choix

Travaux pratiques notés, à réaliser en trois semaines, seul ou en binôme. Choisis un commerce réel ou fictif de ton environnement (boutique, atelier, restaurant, salon, service de réparation…) et réalise son site vitrine complet en suivant la démarche du cours. JALON 1 (fin de semaine 1) — Conception : cahier des charges d'une page (objectif, public, pages, contenus, charte, délai, critères de réussite), arborescence, maquettes fil de fer mobile et ordinateur de chaque page, le tout validé par l'enseignant (ou par le commerçant si le commerce est réel). JALON 2 (fin de semaine

2) — Réalisation : dossier de projet organisé, pages HTML intégrées avec le contenu réel, feuille de style responsive (mobile d'abord), fichier JavaScript avec au minimum le menu mobile et un contrôle de formulaire. JALON 3 (fin de semaine 3) — Livraison : grille de tests d'au moins quinze tests remplie et signée, corrections effectuées, site publié en ligne (GitHub Pages ou équivalent gratuit), soutenance de dix minutes avec démonstration en ligne et bilan. Livrables à déposer : le cahier des charges, les maquettes, le dossier du site, la grille de tests, l'adresse du site en ligne.

✔ Attendus du TP 1
Attendus et critères d'évaluation. Jalon 1 : le cahier des charges tient sur une page mais ne laisse aucun blanc sur les points critiques (action attendue du visiteur, contenus et leur fournisseur, délai) ; les maquettes correspondent page à page à l'arborescence ; la version mobile est dessinée AVANT la version ordinateur. Jalon 2 : aucune erreur au validateur W3C sur les pages ; header et footer identiques partout ; un seul fichier CSS et un seul fichier JS partagés ; images avec attribut alt ; prix, horaires et contacts réels et cohérents entre les pages ; affichage correct à 360 px de large sans défilement horizontal. Jalon 3 : la grille de tests couvre les cinq familles (liens, contenu, affichage, interactions, technique) et ses cases « à corriger » montrent une correction datée ; le site en ligne repasse la grille sans écart avec le local ; la soutenance suit le plan en quatre temps (problème, démarche, démonstration, bilan) dans le temps imparti, avec un secours hors-ligne prêt. Barème indicatif : conception 25 %, réalisation 40 %, tests et livraison 20 %, soutenance 15 %. Les erreurs les plus coûteuses : commencer à coder avant le jalon 1, découvrir la mise en ligne la veille de la soutenance, présenter des captures au lieu du site réel.

Exercices corrigés supplémentaires

Exercice 1

Madame Sangaré tient une boulangerie-pâtisserie à Sébénikoro. Elle te dit : « Les gens ne savent pas qu'on fait des gâteaux sur commande pour les mariages. » Transforme cette phrase en extrait de cahier des charges : objectif du site en une phrase mesurable, page ou section à prévoir, contenu précis qu'elle devra te fournir, et critère de réussite vérifiable à la livraison.

✔ Corrigé
Objectif : « Le site doit permettre à tout visiteur de découvrir l'offre de gâteaux sur commande et de passer commande par téléphone ou WhatsApp en moins de trois clics depuis l'accueil. » Page/section : une page dédiée « Gâteaux sur commande » (ou une section bien visible dès l'accueil), avec galerie de modèles. Contenus à fournir par Mme Sangaré : photos réelles de 8 à 12 gâteaux déjà réalisés, fourchettes de prix selon la taille, délai minimum de commande (par exemple 72 heures), numéro WhatsApp des commandes, exemples de parfums et de tailles. Critère de réussite vérifiable : depuis la page d'accueil, un testeur trouve un modèle de gâteau, son prix indicatif et le bouton WhatsApp en moins de 30 secondes ; et la consigne « commander au moins 72 h à l'avance » est lisible avant le bouton d'action. Remarque : la phrase de départ parlait d'un problème de notoriété ; le cahier des charges la traduit en parcours de visiteur et en contenus — c'est exactement le travail de la leçon 1.

Exercice 2

Pour une boutique de vente de pagnes et de bazin située au Grand Marché, dessine (en texte) l'arborescence d'un site vitrine de quatre pages maximum, puis la maquette fil de fer de la page d'accueil mobile. La boutique veut surtout recevoir des commandes par WhatsApp.

✔ Corrigé

Arborescence possible : Accueil (index.html) ; Nos tissus (tissus.html) — catégories pagnes, bazin riche, bazin moins riche, avec photos et prix au mètre ou à la pièce ; Conseils & modèles (modeles.html) — idées de tenues réalisées avec les tissus de la boutique, qui donnent envie et prouvent le rendu ; Contact (contact.html) — adresse au Grand Marché avec repères, horaires, bouton WhatsApp. Quatre pages suffisent : multiplier les pages disperserait des contenus minces. Maquette mobile de l'accueil : en-tête logo + menu burger ; grande photo d'un bazin éclatant avec le slogan « Des tissus qui racontent Bamako » et un bouton [Commander sur WhatsApp] dès le premier écran — car c'est l'action n° 1 du cahier des charges ; section « Nos tissus » avec trois cartes (photo, nom, prix) menant à tissus.html ; section « Modèles réalisés » avec deux photos de tenues ; témoignage d'une cliente ; bloc pratique (horaires du marché, adresse, téléphone) ; pied de page. Le bouton WhatsApp réapparaît en fin de page : le visiteur ne doit jamais avoir à chercher comment commander.

Exercice 3

Rédige un plan de test minimal mais sérieux (exactement six tests) pour la seule page Contact de « Saveurs du Fleuve », en couvrant obligatoirement : un lien, le formulaire, l'affichage mobile et l'exactitude du contenu. Pour chaque test, précise le geste effectué et le résultat attendu.

✔ Corrigé

Test 1 (lien) : cliquer le numéro de téléphone affiché sur un téléphone réel — attendu : le composeur s'ouvre avec le +223 70 12 34 56 pré-rempli. Test 2 (lien) : cliquer l'adresse ou le lien « itinéraire » s'il existe — attendu : ouverture de la carte sur l'ACI 2000, pas sur une autre ville. Test 3 (formulaire) : envoyer le formulaire entièrement vide — attendu : envoi bloqué et message clair indiquant le champ manquant. Test 4 (formulaire) : saisir un nom, un numéro valide à 8 chiffres et un message, envoyer — attendu : confirmation visible pour l'utilisateur (message de remerciement), aucune erreur dans la console. Test 5 (affichage mobile) : ouvrir la page à 360 px de large — attendu : aucun défilement horizontal, le bouton d'envoi atteignable sans zoom, champs les uns sous les autres. Test 6 (contenu) : comparer caractère par caractère les horaires de la page avec ceux validés par la cliente (11h–23h, tous les jours) et l'adresse avec le cahier des charges — attendu : identité parfaite. Six tests bien choisis valent mieux que trente tests vagues : chacun a un geste, un attendu, un verdict.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Le cahier des charges sert d'abord à…

   A) Choisir les couleurs du site

   B) Écrire par écrit le besoin, les pages, les contraintes et les critères de réussite avant de coder

   C) Remplacer les maquettes

2. Une maquette fil de fer (wireframe) contient…

   A) Uniquement la structure : blocs, titres et emplacements, sans décoration

   B) Les couleurs définitives et les vraies photos

   C) Le code HTML de la page

3. La règle des trois clics signifie que…

   A) Le site doit avoir trois pages

   B) Toute information importante doit être atteignable en au plus trois clics depuis l'accueil

   C) Le visiteur doit cliquer trois fois pour acheter

4. Travailler « mobile d'abord » veut dire…

   A) Ne faire le site que pour les téléphones

   B) Tester sur mobile seulement à la fin

   C) Écrire d'abord les styles pour petit écran, puis enrichir pour les grands écrans

5. Le nom obligatoire de la page d'accueil d'un site statique est…

   A) index.html

   B) accueil.html

   C) home.html

6. Un site est publié sur GitHub Pages mais la mise en forme a disparu. La cause la plus fréquente est…

   A) GitHub Pages ne prend pas en charge le CSS

   B) Le fichier CSS n'a pas été envoyé, ou son chemin/casse ne correspond pas au lien HTML

   C) Le navigateur est trop ancien

7. Lors de la phase de tests, on corrige en priorité…

   A) Les défauts esthétiques, plus rapides à traiter

   B) Les défauts dans l'ordre où on les découvre

   C) Ce qui trompe ou bloque le visiteur (prix faux, lien mort, formulaire cassé)

8. Le contrôle du formulaire en JavaScript sert à…

   A) Guider le visiteur avec un retour immédiat et un message clair avant l'envoi

   B) Empêcher définitivement toute erreur dans les données reçues par le client

   C) Remplacer le cahier des charges

9. Pendant la soutenance, la démonstration doit de préférence se faire…

A) Sur des captures d'écran uniquement

B) En lisant le code source ligne par ligne

C) Sur le site en ligne, avec un parcours de visiteur préparé (et des captures en secours)

10. Dans le bilan d'un projet de site vitrine statique, une limite honnête à annoncer est…

A) Le site ne peut pas avoir plus de quatre pages

B) Les prix ne peuvent être modifiés que dans le code, par le développeur

C) Le site est invisible sur les moteurs de recherche

Corrigés du QCM

1. Réponse B — C'est le document de cadrage : il fige ce qui doit être construit, pour qui, et comment on saura que c'est réussi.

2. Réponse A — Le wireframe sert à discuter la disposition et le contenu, en noir et blanc, avant tout choix esthétique ou technique.

3. Réponse B — C'est une règle de navigation : au-delà de trois clics, la plupart des visiteurs abandonnent.

4. Réponse C — Les styles de base servent le mobile ; les requêtes média en min-width ajoutent les dispositions larges.

5. Réponse A — Les hébergeurs, dont GitHub Pages, servent automatiquement index.html à la racine du site.

6. Réponse B — En local, la casse des noms est souvent ignorée ; sur le serveur, style.css et Style.css sont deux fichiers différents.

7. Réponse C — La gravité décide : tromper ou bloquer le visiteur passe avant l'alignement d'un bloc.

8. Réponse A — C'est un confort de saisie côté navigateur : utile et attendu, mais il ne constitue pas une garantie absolue.

9. Réponse C — Montrer le site réel qui fonctionne est la preuve la plus forte ; les captures ne servent que de plan B.

10. Réponse B — Sans outil de gestion de contenu, toute mise à jour passe par l'édition des fichiers : c'est LA limite structurelle du statique, à compenser par une procédure claire.

Résumé — les notions importantes

• La démarche professionnelle suit un ordre fixe : besoin, conception, réalisation, tests, livraison — jamais l'inverse.

• Le cahier des charges transforme une demande floue en pages, contenus, contraintes, délai et critères de réussite mesurables.

• L'arborescence organise les pages (règle des 3 clics) ; les maquettes fil de fer valident la structure, mobile d'abord, avant le code.

• L'intégration repose sur un dossier propre, un header et un footer communs, une seule feuille CSS et un seul fichier JS partagés.

• Le responsive se construit mobile d'abord : variables de charte, Flexbox/Grid, requêtes média en min-width, aucun défilement horizontal.

• L'interactivité reste utile et robuste : menu mobile, année automatique, validation de formulaire — le site vitrine fonctionne aussi sans JavaScript.

• Grille de tests écrite, corrections par gravité, retest, mise en ligne gratuite (GitHub Pages), retest en ligne, puis soutenance préparée avec démonstration réelle et bilan honnête.

Fiches de révision express

Chaque fiche reprend les points « À retenir » d'une leçon. Relis-les la veille d'un contrôle : cinq minutes par fiche suffisent.

Fiche 1 — Rédiger le cahier des charges d'un site vitrine
On code après avoir compris le besoin, jamais avant : l'entretien client précède toujours l'éditeur de texte. ; Un objectif flou (« un beau site ») ne se teste pas ; un critère mesurable (« numéro visible en 3 clics ») si. ; Le cahier des charges liste aussi ce que le CLIENT doit fournir (photos, textes, prix) et pour quand.

Fiche 2 — Arborescence et maquettes des pages
Arborescence = quelles pages, quels liens ; maquette = quels blocs, à quelle place, dans chaque page. ; Une maquette fil de fer ne contient ni couleurs ni vraies photos : on discute la structure, pas les goûts. ; Dessiner la version mobile en premier : c'est l'écran réel de la plupart des visiteurs.

Fiche 3 — Intégration HTML des différentes pages
index.html est la porte d'entrée du site ; les autres pages portent des noms courts, sans espaces ni accents. ; Header, navigation et footer identiques sur toutes les pages : la stabilité rassure le visiteur. ; Contenu réel tout de suite (vrais textes, vrais prix) : jamais de lorem ipsum dans une livraison.

Fiche 4 — Mise en forme CSS et adaptation aux mobiles
Mobile d'abord : styles de base pour petit écran, enrichissements en min-width pour les grands. ; Les variables CSS dans :root rendent la charte modifiable en un seul endroit. ; Les trois contrôles express : pas de dépassement d'image, pas de défilement horizontal, boutons assez grands.

Fiche 5 — Touches d'interactivité en JavaScript
Un seul fichier script.js partagé par toutes les pages : corriger une fois, corriger partout. ; Tester les cas limites du formulaire (vide, trop court, espaces) : c'est là que vivent les erreurs. ; Si JavaScript casse, le site doit rester utilisable : l'interactivité est un bonus, pas une condition.

Fiche 6 — Tests, corrections et mise en ligne du site
« Ça marche chez moi » n'est pas un test : la grille écrite, exécutée case par case, l'est. ; Corriger par gravité décroissante : bloquant d'abord, trompeur ensuite, esthétique en dernier. ; Le site en ligne se reteste entièrement : fichiers oubliés et casses de noms n'apparaissent que là.

Fiche 7 — Présentation et soutenance du projet
Préparer, chronométrer, répéter à voix haute : une soutenance se répète comme un examen, pas comme une conversation. ; Démonstration sur le site en ligne, parcours de visiteur préparé, captures de secours prêtes. ; Annoncer soi-même les limites du projet et ses évolutions possibles : c'est une preuve de maturité, pas un aveu de faiblesse.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — Apprendre HTML & CSS de zéro – Jour 1/30 : comment fonctionne un site web + installer VS Code

Chaîne : Jean-Luc | Developer · Langue : Français · Niveau : Débutant
Avant de commencer le projet : comprendre comment un site web fonctionne (navigateur, fichiers, rôles de HTML, CSS et JavaScript) et installer l'éditeur VS Code avec les extensions utiles. Le point de départ idéal pour organiser son poste de travail comme dans la leçon 1.

https://www.youtube.com/watch?v=YGdvw3V2LU4

Leçon 2 — FIGMA : Apprendre à faire une maquette de site web - Tutoriel facile en français

Chaîne : Coriace | No-Code · Langue : Français · Niveau : Débutant
Un tutoriel pas à pas pour construire une maquette de site dans Figma, de la structure des blocs à une présentation propre et prête à intégrer. Parfait pour transformer tes maquettes papier de la leçon 2 en maquettes numériques présentables à un client.

https://www.youtube.com/watch?v=tXPTymE828U

Leçon 3 — Tutoriel Debutant - Créer son premier site avec HTML / CSS

Chaîne : Graven - Développement · Langue : Français · Niveau : Débutant
Un projet complet réalisé à partir d'une maquette : création du dossier, page d'accueil, menu de navigation, sections de services et de produits, pied de page, puis version responsive. C'est exactement la démarche d'intégration de la leçon 3, en vraie grandeur.

https://www.youtube.com/watch?v=6hCGTJCo_Uo

Leçon 3 — Créer un site de A à Z - HTML5, CSS3 & Bootstrap - TP [FR]

Chaîne : Code & Design · Langue : Français · Niveau : Débutant à intermédiaire
Un travaux pratiques filmé qui mobilise toutes les compétences HTML et CSS pour construire un site complet d'une page. Utile pour comparer ta méthode d'intégration à une autre démarche complète et prendre des idées d'organisation.

https://www.youtube.com/watch?v=vSeW6_0SMsk

Leçon 6 — J'ai créé mon Portfolio avec l'IA en 10 min — GitHub Pages + Supabase (GRATUIT)

Chaîne : CodeAvecAbel · Langue : Français · Niveau : Débutant
La partie qui nous intéresse : la mise en ligne gratuite d'un site statique avec GitHub Pages, de la création du dépôt à l'adresse publique du site. Regarde-la en suivant la recette de la leçon 6 sur ton propre projet.

https://www.youtube.com/watch?v=mZ_Vk_9XvAM

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• MDN Web Docs (developer.mozilla.org/fr) — références HTML, CSS et JavaScript, guides « responsive design » et « formulaires ».

• Documentation officielle GitHub Pages (docs.github.com/fr/pages) — création de dépôt, publication et dépannage des sites statiques.

• Validateur HTML du W3C (validator.w3.org) et outils de développement des navigateurs (onglet Réseau) pour la phase de tests.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !